



American International University-Bangladesh (AIUB)
Department of computer science
Faculty of Science & Technology (FST)

Semester: FALL 2025-26
Course: Introduction to Data Science
Section : K
Group: 13

Project Title:
Student Habits Analysis, Data Cleaning and Exploratory Data Analysis

Dataset Title: Student Habits Performance

Supervised By:
Kamrun Naher Koli
Lecturer,
Department of Computer Science
Faculty of Science and Technology
American International University-Bangladesh

Submitted By :

Name	ID
Avoy Mollick	23-50066-1
Arpon Mollick	23-50061-1
MD. ASHIQ BIN HAMID	23-53342-3

Submission Date : 08-04-2026

1. Introduction

This project focuses on data preprocessing, cleaning, and exploratory data analysis using the R programming language. The dataset contains student habit and performance data, including attributes such as study hours, social media usage, sleep patterns, attendance, and exam scores, to analyze how different lifestyle factors influence academic performance.

The main objective of this project is to:

- Identify and handle missing values
- Detect and remove outliers
- Normalize data
- Remove duplicate records
- Balance the dataset
- Perform descriptive statistical analysis

The project follows a structured pipeline to ensure the dataset is properly prepared for analysis.

2. Dataset Description

The dataset used in this project is titled “**Student Habit and Performance Dataset**”, which contains detailed information about students’ daily habits and their academic outcomes. Each row represents an individual student along with various attributes related to lifestyle, behavior, and performance. The dataset includes both numerical and categorical variables, enabling comprehensive analysis of how different factors influence exam results.

Key Attributes

- **Age:** Age of the student in years.
- **Gender:** Gender of the student (Male, Female, Other).
- **Study_Hours_per_Day:** Number of hours a student studies per day.
- **Social_Media_Hours:** Time spent on social media per day.
- **Netflix_Hours:** Time spent watching Netflix or entertainment platforms.
- **Attendance_Percentage:** Percentage of classes attended by the student.
- **Sleep_Hours:** Number of hours the student sleeps per day.
- **Exercise_Frequency:** Frequency of physical exercise (e.g., number of times per week).
- **Mental_Health_Rating:** Self-reported mental health score of the student.
- **Extracurricular_Participation** – Participation in activities (Yes/No)
- **Exam_Score (Target):** Final exam performance score of the student.

The dataset contains a mix of numerical and categorical variables, making it suitable for preprocessing and analysis. Overall, it provides a comprehensive view of student behavior and performance, allowing the application of various data science techniques such as data cleaning, handling missing values, outlier detection, normalization, and correlation analysis.

3. Project Implementation

Task 1: Setup Libraries

Code

```
packages <- c("ggplot2", "dplyr", "tidyr", "scales", "GGally", "e1071", "caret")

for (pkg in packages) {
  if (!require(pkg, character.only = TRUE)) {
    install.packages(pkg)
    library(pkg, character.only = TRUE)
  }
}
```

Output

```
Loading required package: ggplot2
Loading required package: dplyr

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

  filter, lag

The following objects are masked from 'package:base':

  intersect, setdiff, setequal, union

Loading required package: tidyr
Loading required package: scales
Loading required package: GGally
Loading required package: e1071
```

Interpretation

These libraries are used for data manipulation, reading datasets, handling missing values, and visualization.

Data Visualization (ggplot2): Used to create graphs such as histograms, boxplots, and scatter plots for understanding data patterns.

Data Manipulation (dplyr & tidyr): Used to clean, filter, and organize the dataset, including handling missing values and reshaping data.

Data Scaling (scales): Helps format and adjust values in visualizations for better readability.

Visualization Enhancement (GGally): Used for advanced plots like correlation matrices.

Statistical Analysis (e1071): Provides functions to analyze data distribution.

Modeling (caret): Used for splitting the dataset into training and testing sets and supporting model preparation.

Task 2: Load and Inspect Data

Code

```
student_habit <- read.csv(file.choose())

# Preview
head(student_habit)
str(student_habit)

num_cols <- c("age", "study_hours_per_day", "social_media_hours", "netflix_hours",
              "attendance_percentage", "sleep_hours", "exercise_frequency",
              "mental_health_rating", "exam_score")

for(col in num_cols){
  student_habit[[col]] <- as.numeric(as.character(student_habit[[col]]))
}
```

Output

```
'data.frame': 1007 obs. of 16 variables:
 $ student_id      : chr "S1000" "S1001" "S1002" "S1003" ...
 $ age             : chr "23" "20" "21" "23" ...
 $ gender          : chr "Female" "Female" "Male" "Female" ...
 $ study_hours_per_day : num 0 6.9 1.4 1 5 7.2 5.6 221 4.4 4.8 ...
 $ social_media_hours : num 1.2 2.8 3.1 3.9 4.4 1.3 1.5 1 2.2 122 ...
 $ netflix_hours    : num 1.1 2.3 1.3 1 0.5 0 1.4 -9 1.7 1.3 ...
 $ part_time_job    : chr "No" "No" "No" "No" ...
 $ attendance_percentage : chr "85" "97.3" "94.8" "71" ...
 $ sleep_hours      : num 8 4.6 25 9.2 4.9 7.4 6.5 4.6 7.1 7.5 ...
 $ diet_quality     : chr "Fair" "4324" "Poor" "Poor" ...
 $ exercise_frequency : int 6 6 1 4 3 1 2 0 333 5 ...
 $ parental_education_level : chr "Master" "High School" "High School" "Master" ...
 $ internet_quality  : chr "Average" "Average" "" "Good" ...
 $ mental_health_rating : chr "8" "8" "1" "1" ...
 $ extracurricular_participation : chr "Yes" "No" "No" "Yes" ...
 $ exam_score       : chr "56.2" "100" "34.3" "26.8" ...
```

Interpretation

Data Loading and Preview:

The dataset is loaded into the R environment using `read.csv()`, allowing the user to select the file manually. The `head()` function displays the first few rows to get an initial understanding of the data, while `str()` shows the structure and data types of each column.

Numeric Conversion:

Selected columns that should contain numerical values are explicitly converted to numeric format. This ensures consistency in data types and allows proper statistical analysis. Any non-numeric or invalid values are automatically converted to NA, helping identify data quality issues.

Task 3: DETECT DATA ISSUES

Code

```
# 3.1 Missing values (including blank, space, NA, ?, null)
student_habit[student_habit == "" | student_habit == " " | student_habit == "NA" | student_habit == "?"] <- NA

cat("Missing values in each column:\n")
print(colSums(is.na(student_habit)))

# 3.2 Duplicate rows
dup_rows <- sum(duplicated(student_habit))
cat("Duplicate rows found:", dup_rows, "\n")

# 3.3 Inconsistent categories
cat("Unique values in Gender column:\n")
print(unique(student_habit$gender))

cat("Unique values in Part Time Job column:\n")
print(unique(student_habit$part_time_job))

# 3.4 Invalid numeric values
num_cols <- names(student_habit)[sapply(student_habit, is.numeric)]

cat("Invalid numeric values check:\n")
for(col in num_cols){
  cat(col, ":\n")
  # Use proper thresholds
  if(col %in% c("study_hours_per_day", "social_media_hours", "netflix_hours", "sleep_hours")){
    cat(" Negative values:", sum(student_habit[[col]] < 0, na.rm = TRUE), "\n")
    cat(" values >", 24, ":", sum(student_habit[[col]] > 24, na.rm = TRUE), "\n\n")
  } else if(col == "exercise_frequency"){
    cat(" Negative values:", sum(student_habit[[col]] < 0, na.rm = TRUE), "\n")
    cat(" values >", 7, ":", sum(student_habit[[col]] > 7, na.rm = TRUE), "\n\n")
  } else if(col %in% c("age", "attendance_percentage", "exam_score")){
    cat(" Negative values:", sum(student_habit[[col]] < 0, na.rm = TRUE), "\n")
    cat(" values >", 100, ":", sum(student_habit[[col]] > 100, na.rm = TRUE), "\n\n")
  } else if(col == "mental_health_rating"){
    cat(" Negative values:", sum(student_habit[[col]] < 0, na.rm = TRUE), "\n")
    cat(" values >", 10, ":", sum(student_habit[[col]] > 10, na.rm = TRUE), "\n\n")
  } else {
    cat(" Negative values:", sum(student_habit[[col]] < 0, na.rm = TRUE), "\n\n")
  }
}
```

Output

```
Missing values in each column:
      student_id      age      gender
      0           4           3
study_hours_per_day  social_media_hours  netflix_hours
      6           0           1
part_time_job      attendance_percentage  sleep_hours
      2           1           1
diet_quality      exercise_frequency  parental_education_level
      2           2           1
internet_quality  mental_health_rating  extracurricular_participation
      2           6           2
exam_score
      1
```

```
Duplicate rows found: 7
Unique values in Gender column:
[1] "Female" "Male" "Other" NA "Others" "M0le" "Females"
Unique values in Part Time Job column:
[1] "No" "Yes" "maybe" "oooo" "eerr" "rwe" "wre" "werwwe" "wr"
[10] NA "ee"
Invalid numeric values check:
age :
Negative values: 0
Values > 100 : 6

study_hours_per_day :
Negative values: 1
Values > 24 : 6

social_media_hours :
Negative values: 7
Values > 24 : 1

netflix_hours :
Negative values: 6
Values > 24 : 2

attendance_percentage :
Negative values: 3
Values > 100 : 5

sleep_hours :
Negative values: 0
Values > 24 : 6

exercise_frequency :
Negative values: 0
Values > 7 : 8

mental_health_rating :
Negative values: 0
Values > 10 : 0

exam_score :
Negative values: 3
Values > 100 : 6
```

Interpretation

This section is used to identify problems in the dataset before cleaning and analysis. Detecting issues ensures data quality and reliability.

Missing Values:

```
# 3.1 Missing values (including blank, space, NA, ?, null)
student_habit[student_habit == "" | student_habit == " " | student_habit == "NA" | student_habit == "?"] <- NA

cat("Missing values in each column:\n")
print(colSums(is.na(student_habit)))
```

Explanation:

- Replaces blank entries, spaces, "NA", or "?" with proper NA.
- Counts missing values in each column.

Interpretation:

- Helps identify incomplete data that needs imputation before analysis.
- For example, if sleep_hours has 3 missing values, we know it must be replaced with mean or median.

Duplicate Rows:

```
# 3.2 Duplicate rows
dup_rows <- sum(duplicated(student_habit))
cat("Duplicate rows found:", dup_rows, "\n")
```

Explanation:

Checks for rows that are exactly identical.

Interpretation:

- Duplicate entries can bias analysis.
- If `dup_rows = 2`, those rows should be removed to avoid skewing results.

Inconsistent Categories:

```
# 3.3 Inconsistent categories
cat("Unique values in Gender column:\n")
print(unique(student_habit$gender))

cat("Unique values in Part Time Job column:\n")
print(unique(student_habit$part_time_job))
```

Explanation:

Lists all unique values in categorical columns like gender and part_time_job.

Interpretation:

- Identifies inconsistent entries such as "Female" vs "female" or "maybe" in part_time_job.
- These need standardization for accurate analysis.

Invalid Numeric Values:

```
# 3.4 Invalid numeric values
num_cols <- names(student_habit)[sapply(student_habit, is.numeric)]

cat("Invalid numeric values check:\n")
```

Explanation:

- Iterates over numeric columns to detect:
 - Negative values where not possible (e.g., study hours < 0)
 - Unrealistic maximums (e.g., study_hours > 24, age > 100, mental_health_rating > 10)

Interpretation:

- Highlights data errors that must be corrected.
- For example, study_hours_per_day = 221 is impossible and must be fixed or replaced with mean.

Task 4: DATA CLEANING

Code

Remove duplicates

```
# 4.1 Remove duplicates
dup_count <- sum(duplicated(student_habit))
cat("Duplicate rows found:", dup_count, "\n")

student_habit <- unique(student_habit)
cat("Duplicates removed\n")
```

OUTPUT:

```
Duplicate rows found: 7
Duplictes removed
```

Interpretation:

- duplicated() identifies repeated rows, returning TRUE for each duplicate.
- sum() counts how many duplicates exist.
- unique() removes all duplicates, keeping only the first occurrence of each row.
- This ensures each observation is distinct, which is critical for accurate statistical calculations.

Handle Missing Values

```
cat("Handling Missing Values...\n")

get_mode <- function(x) {
  ux <- na.omit(unique(x))
  ux[which.max(tabulate(match(x, ux)))]
}

for(col in names(student_habit)) {
  if(is.numeric(student_habit[[col]])) {
    na_count <- sum(is.na(student_habit[[col]]))
    if(na_count > 0){
      cat("Replacing", na_count, "missing values in", col, "with mean\n")
      student_habit[[col]][is.na(student_habit[[col]])] <- mean(student_habit[[col]], na.rm = TRUE)
    }
  } else {
    na_count <- sum(is.na(student_habit[[col]]))
    if(na_count > 0){
      cat("Replacing", na_count, "missing values in", col, "with mode\n")
      student_habit[[col]][is.na(student_habit[[col]])] <- get_mode(student_habit[[col]])
    }
  }
}
```

OUTPUT:

```
Handling Missing Values...
Replacing 4 missing values in age with mean
Replacing 3 missing values in gender with mode
Replacing 6 missing values in study_hours_per_day with mean
Replacing 1 missing values in netflix_hours with mean
Replacing 2 missing values in part_time_job with mode
Replacing 1 missing values in attendance_percentage with mean
Replacing 1 missing values in sleep_hours with mean
Replacing 2 missing values in diet_quality with mode
Replacing 2 missing values in exercise_frequency with mean
Replacing 1 missing values in parental_education_level with mode
Replacing 2 missing values in internet_quality with mode
Replacing 6 missing values in mental_health_rating with mean
Replacing 2 missing values in extracurricular_participation with mode
Replacing 1 missing values in exam_score with mean
```

Interpretation:

- get_mode() defines a function to find the most frequent value in a vector, ignoring NAs.
- The for loop iterates through all columns.
- is.numeric() checks if the column is numeric; if so, NA values are replaced by the **mean**.
- For categorical columns, NAs are replaced by the **mode**.
- This loop automates missing value handling for any column type without manual specification.

Fix Invalid Numeric Values

```
cat("Fixing Invalid Numeric Values...\n")

for(col in num_cols){
  # Define upper limits
  upper_limit <- ifelse(col %in% c("study_hours_per_day", "social_media_hours", "netflix_hours", "sleep_hours"), 24,
                        ifelse(col == "exercise_frequency", 7,
                              ifelse(col %in% c("age", "attendance_percentage", "exam_score"), 100,
                                    ifelse(col == "mental_health_rating", 10, Inf))))

  exceed_count <- sum(student_habit[[col]] > upper_limit, na.rm = TRUE)
  if(exceed_count > 0){
    cat("Column", col, "has", exceed_count, "values exceeding", upper_limit, ". Replacing with mean\n")
    student_habit[[col]][student_habit[[col]] > upper_limit] <- mean(student_habit[[col]], na.rm = TRUE)
  }

  neg_count <- sum(student_habit[[col]] < 0, na.rm = TRUE)
  if(neg_count > 0){
    cat("Column", col, "has", neg_count, "invalid negative values. Replacing with mean\n")
    student_habit[[col]][student_habit[[col]] < 0] <- mean(student_habit[[col]], na.rm = TRUE)
  }
}
```

OUTPUT:

```
Fixing Invalid Numeric Values...
Column age has 6 values exceeding 100 . Replacing with mean
Column study_hours_per_day has 6 values exceeding 24 . Replacing with mean
Column study_hours_per_day has 1 invalid negative values. Replacing with mean
Column social_media_hours has 1 values exceeding 24 . Replacing with mean
Column social_media_hours has 7 invalid negative values. Replacing with mean
Column netflix_hours has 2 values exceeding 24 . Replacing with mean
Column netflix_hours has 6 invalid negative values. Replacing with mean
Column attendance_percentage has 5 values exceeding 100 . Replacing with mean
Column attendance_percentage has 3 invalid negative values. Replacing with mean
Column sleep_hours has 7 values exceeding 24 . Replacing with mean
Column exercise_frequency has 10 values exceeding 7 . Replacing with mean
Column exam_score has 6 values exceeding 100 . Replacing with mean
Column exam_score has 2 invalid negative values. Replacing with mean
```

Interpretation:

- `ifelse()` dynamically sets upper thresholds for different numeric columns.
- `sum(... > upper_limit)` counts how many values exceed the valid range.
- Values exceeding the threshold or below 0 are replaced with the **column mean**.
- This logic standardizes numeric data by correcting out-of-range values programmatically.

Fix Inconsistent Text Values

```
# Gender: only allow "male", "female", "other"
student_habit$gender <- tolower(student_habit$gender)
student_habit$gender[!(student_habit$gender %in% c("male", "female", "other"))] <- "other"

# Part-time job: only allow "yes", "no"
student_habit$part_time_job <- tolower(student_habit$part_time_job)
student_habit$part_time_job[!(student_habit$part_time_job %in% c("yes", "no"))] <- "yes"
print(unique(student_habit$gender))
print(unique(student_habit$part_time_job))
cat("Data cleaning completed\n")
```

OUTPUT:

```
Fixing inconsistent text values...
[1] "female" "male"   "other"
[1] "no"    "yes"
Data cleaning completed
```

Interpretation:

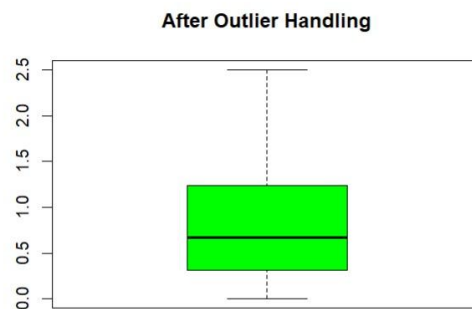
- `tolower()`: Converts all text to lowercase to remove inconsistencies like "Female" vs "female".

- **Logical indexing** `!(... %in% ...)`: Identifies values that do **not** belong to the allowed categories.
- **Assignment** `<=`: Replaces all invalid entries with a **default valid category** ("other" for gender, "yes" for part-time job).
- This ensures that **categorical variables contain only consistent, allowed values**, which is crucial for encoding and analysis.
- The `cat()` statements only print **progress messages**, not the actual changes in the dataset.

Data pre-processing:

Outlier handle:

```
>
> for(col in num_cols){
+   Q1 <- quantile(student_habit[[col]], 0.25)
+   Q3 <- quantile(student_habit[[col]], 0.75)
+   IQR_val <- Q3 - Q1
+
+   lower <- Q1 - 1.5 * IQR_val
+   upper <- Q3 + 1.5 * IQR_val
+
+   student_habit[[col]] <- ifelse(student_habit[[col]] < lower, lower,
+                                 ifelse(student_habit[[col]] > upper, upper, student_habit[[col]]))
+ }
> cat("Outliers handled\n")
Outliers handled
>
```



After applying the IQR-based capping method, the extreme values were reduced. The boxplot now shows a more balanced distribution without significant outliers.

Encoding:

```
>
> cat_cols <- names(student_habit)[sapply(student_habit, function(x) is.character(x) | is.factor(x))]
>
> for(col in cat_cols){
+   student_habit[[col]] <- as.numeric(factor(student_habit[[col]]))
+ }
>
```

Categorical variables such as gender and other textual attributes were converted into numerical format using encoding techniques. This allows the data to be used in statistical analysis and machine learning models.

Normalization:

```
>
> student_habit[num_cols] <- lapply(student_habit[num_cols], function(x){
+   (x - mean(x)) / sd(x)
+ })
>
> cat("Normalization done\n")
Normalization done
>
```

Z-score normalization was applied to numerical variables to standardize the data. This ensures that all features have a mean of 0 and standard deviation of 1, improving comparability across variables.

```

>
> set.seed(123)
>
> train_index <- createDataPartition(student_selected$exam_score, p=0.8, list=FALSE)
>
> train_data <- student_selected[train_index, ]
> test_data <- student_selected[-train_index, ]
>
> cat("Train rows:", nrow(train_data), "\n")
Train rows: 805
> cat("Test rows:", nrow(test_data), "\n")
Test rows: 200

```

The dataset was divided into training and testing sets using an 80:20 ratio. This helps in evaluating the performance of models and ensures that the analysis is not biased.

```

> summary(student_habit)
 student_id      age      gender      study_hours_per_day  social_media_hours  netflix_hours  part_time_job
Length:1000    Min.   : 1.00    Length:1000    Min.   :0.000    Min.   :0.000    Min.   : 0.000    Length:1000
Class :character 1st Qu.:18.00    Class :character 1st Qu.:2.600    1st Qu.:1.700    1st Qu.: 1.000    Class :character
Mode  :character Median :20.00    Mode  :character Median :3.500    Median :2.500    Median : 1.800    Mode  :character
                Mean  :20.75                Mean :3.604        Mean :2.506        Mean : 1.852
                3rd Qu.:23.00                3rd Qu.:4.600      3rd Qu.:3.300      3rd Qu.: 2.525
                Max.  :49.78                Max.  :8.300        Max.  :7.200        Max. :13.128

 attendance_percentage  sleep_hours  diet_quality  exercise_frequency  parental_education_level  internet_quality
Min.   : 0.00          Min.   : 3.20    Length:1000    Min.   : 0.000    Length:1000    Length:1000
1st Qu.: 78.17          1st Qu.: 5.60    Class :character 1st Qu.: 1.000    Class :character  Class :character
Median : 84.50          Median : 6.50    Mode  :character Median : 3.000    Mode  :character  Mode  :character
Mean   : 84.14          Mean  :11.18                Mean : 3.132
3rd Qu.: 91.00          3rd Qu.: 7.30                3rd Qu.: 5.000
Max.   :100.00          Max.  :679.90                Max.  :12.144

 mental_health_rating  extracurricular_participation  exam_score
Min.   : 1.000          Length:1000    Min.   : 18.40
1st Qu.: 3.000          Class :character 1st Qu.: 58.48
Median : 5.000          Mode  :character Median : 70.50
Mean   : 5.426                                Mean : 69.65
3rd Qu.: 8.000                                3rd Qu.: 81.83
Max.   :10.000                                Max.  :100.00

```

The summary statistics provide an overview of the dataset, including mean, median, and range of variables. It shows that students study around 3.5 hours daily and have an average exam score of approximately 69.

Exploratory data analysis:

Exploratory Data Analysis (EDA) was performed to understand the relationships between variables and identify patterns in the dataset. Various graphical techniques were used to visualize the distribution and interaction of key features.

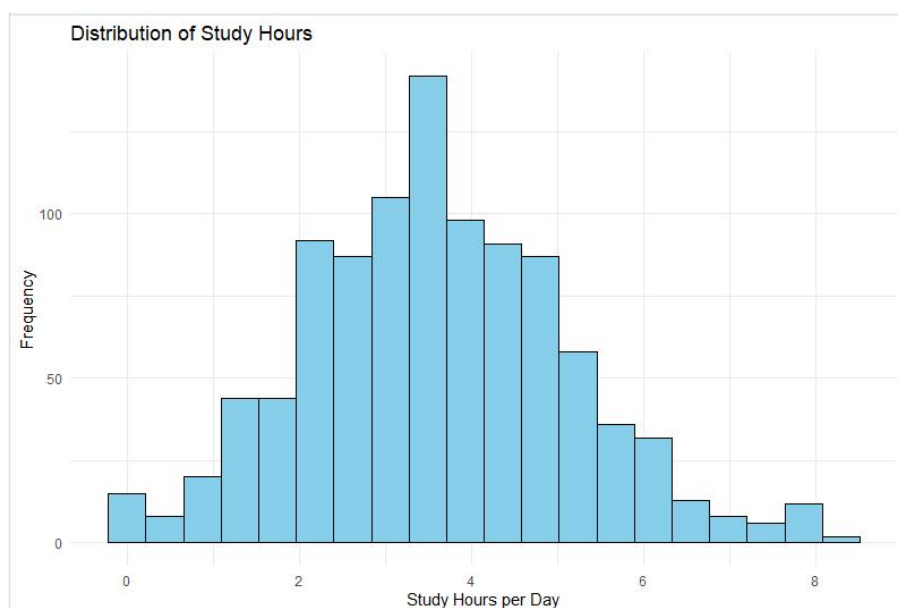


Fig 1: histogram of study hours per day

The histogram shows the distribution of study hours per day. Most students study between 2 to 5 hours, indicating a moderate study pattern among the majority. The distribution appears approximately normal with slight variation.

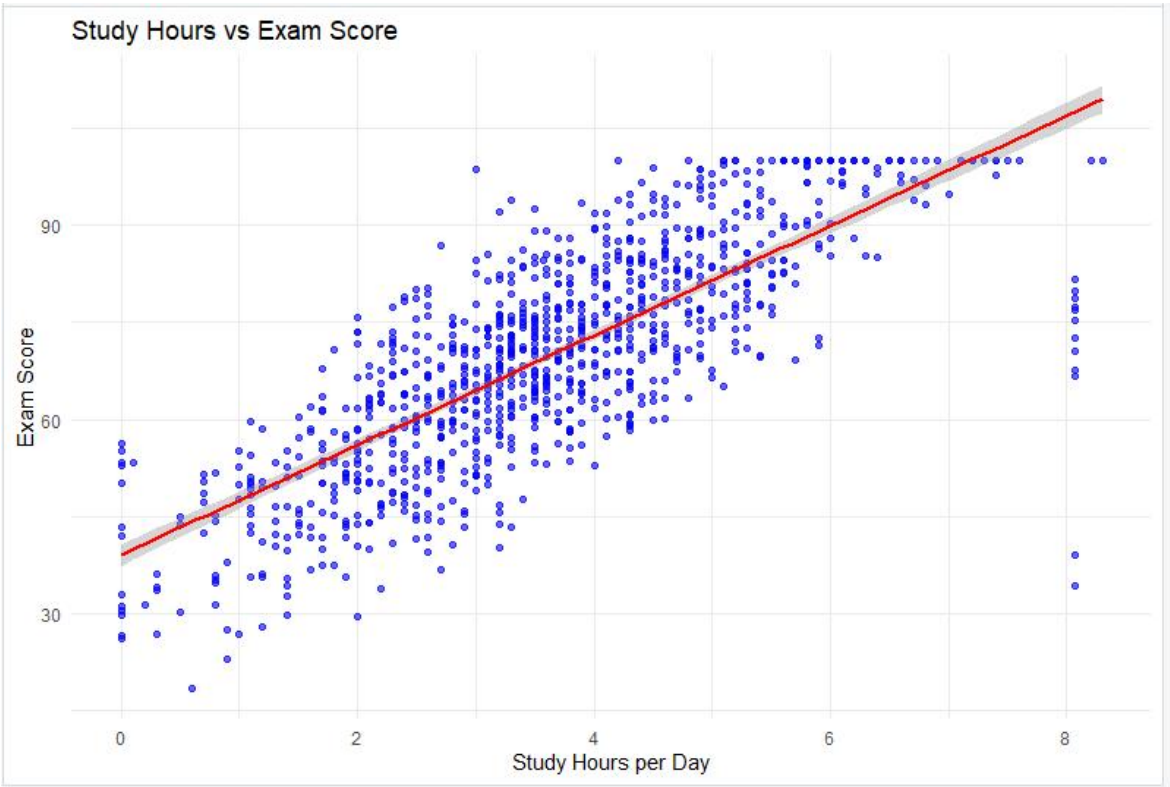


Fig 2: Scatter plot of study hours per day

The scatter plot shows a strong positive relationship between study hours and exam score, indicating that increased study time leads to better academic performance.

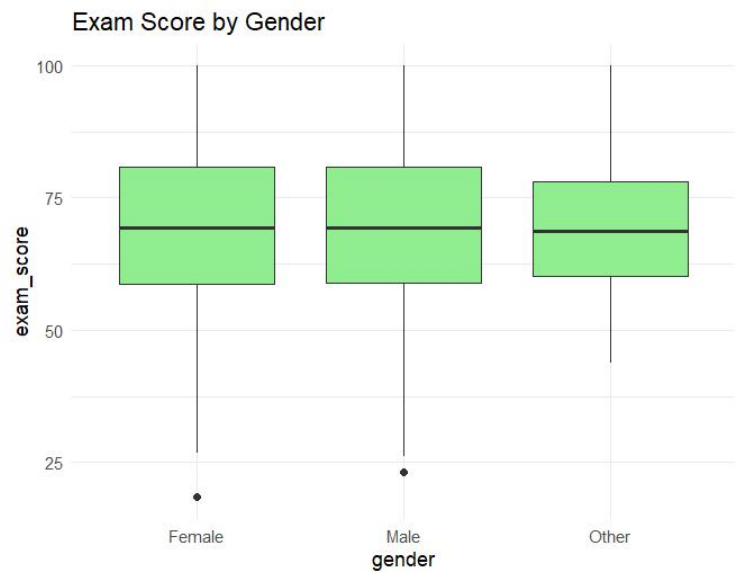


Fig 3: bar chart distribution of students across different genders

The bar chart shows the distribution of students across different genders. The dataset is relatively balanced between male and female students, while a smaller proportion belongs to the “Other” category.

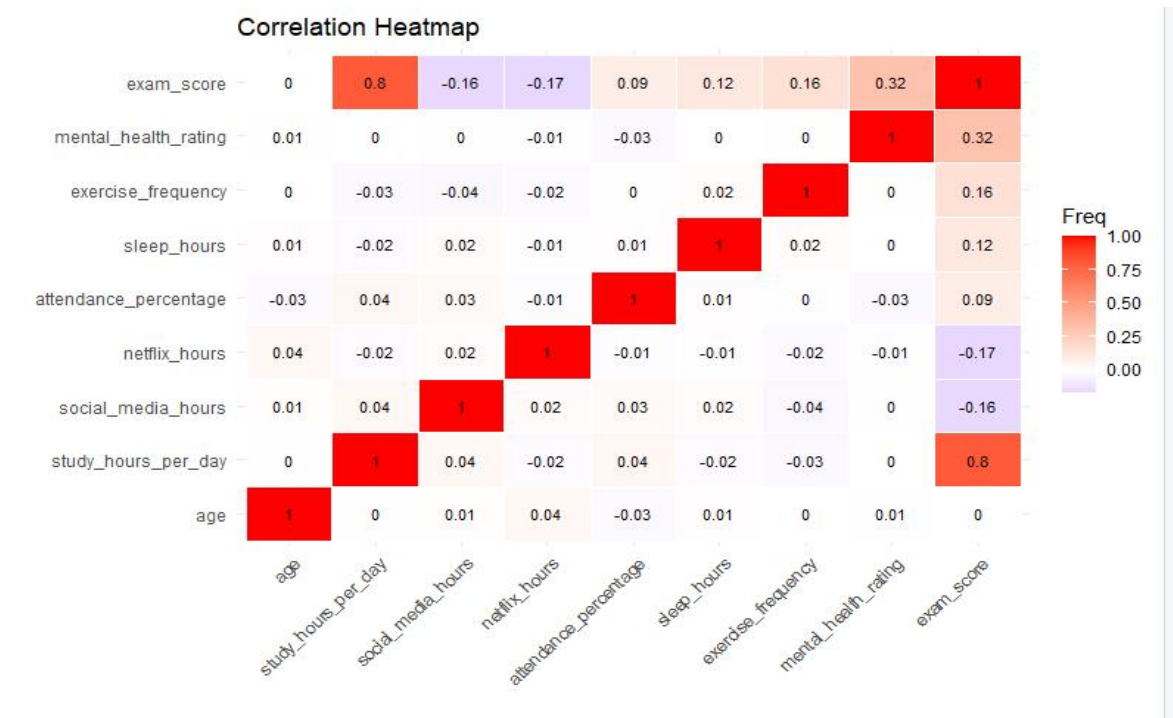


Fig 4: correlation heatmap of numerical variables

The correlation heatmap displays the relationships between numerical variables. Study hours per day show the strongest positive correlation (approximately 0.8) with exam score, indicating that increased study time significantly improves academic performance. Other variables such as social media and Netflix usage show weaker or slightly negative relationships.

Overall, the analysis indicates that study habits, particularly study hours, play a significant role in determining academic performance, while other lifestyle factors have a relatively smaller impact.

Reference:

[1] ahmednasra1, “student_habits_performance,” *Kaggle.com*, Apr. 25, 2025. <https://www.kaggle.com/code/ahmednasra1/student-habits-performance> (accessed Apr. 08, 2026).